

Logo

a Brand company

Version control®

Release 1.0
Issue 1, June 2026
Prepared as a documentation portfolio sample

Includes:

Bitbucket Configuration

- Git
- Bitbucket
- Oxygen XML Author

User Guide

Table of Contents

1	Introduction	7
1.1	About This Guide	7
1.2	Audience.....	7
1.3	Prerequisites	7
2	Getting Started	8
3	Architecture: How the Three Components Interoperate	9
3.1	Rationale for a Git-Based Documentation Workflow	9
3.2	Rationale for Using Oxygen Native Git Integration	9
4	Initial Configuration: Connecting Oxygen to Bitbucket.....	11
4.1	Generate Bitbucket Credentials.....	11
4.2	Register Credentials in Oxygen.....	11
5	Cloning a Documentation Repository.....	12
5.1	Clone via the Git Staging View.....	12
5.2	Open the Project.....	12
6	Standard Editing Workflow	13
6.1	Pull Latest Changes.....	13
6.2	Edit Content.....	13
6.3	Review Modifications.....	13
6.4	Stage and Commit.....	13
6.5	Push to Bitbucket.....	14
7	Branching Strategy for Documentation Changes.....	15
7.1	Create a Branch in Oxygen.....	15
7.2	Reference Branching Model	15
8	Pull Request Review in Bitbucket	16
8.1	Open a Pull Request.....	16
8.2	Review Process.....	16
8.3	Merge.....	16
9	Conflict Resolution in XML Files	17
9.1	Identifying a Conflict.....	17
9.2	Resolution Procedure	17
10	Operational Best Practices.....	18
11	Quick Reference.....	19
12	Glossary	20

Contact Information

Knowledge Creation

For comments or suggestion about this documentation, contact us at:

Email: knowledge.creation@logo.com

Technical Assistance Center

For technical assistance, contact the Logo technical assistance center (TAC) at:

Phone: 1-900-xxx-38888

Knowledge Delivery

For information about product training, contact us at:

Email: knowledge.delivery@logo.com

Sales

This documentation is supplied as a service to our customers.

For sales and general information, contact us at:

Phone: 1-900-xxx-38888

Trademarks and Disclaimers

Trademarks

Logo is a trademark of Logo Inc. in the United States and other countries.

Git® is a registered trademark of Software Freedom Conservancy Inc. in the United States and other countries.

Bitbucket® is a registered trademark of Atlassian Pty Ltd in the United States and other countries.

Oxygen XML Author® is a registered trademark of Syncro Soft in the United States and other countries.

General Disclaimer

Logo and Logo Customer Use Only: The statement provided herein are for informational purpose only and may be amended or altered by Logo without notice or liability. No part of the document may be copied or reproduced without the express written consent of Logo

Document Change Notice

Location of Change	Description of Change
Introduction	▪ Updated following sections: • About This Guide • Prerequisites ▪ Added following sections: • Audience

1 Introduction

In this chapter:

[1.1 About This Guide](#)

[1.2 Audience](#)

[1.3 Prerequisites](#)

1.1 About This Guide

This guide introduces the core concepts and everyday workflows of GitHub, the web-based platform built around the Git version control system. It is written for people who are new to GitHub: students, junior developers, or anyone who needs to contribute to a code repository for the first time.

By the end of this guide, you will be able to create a repository, make and track changes using commits, work with branches, and open a pull request to propose your changes for review.

1.2 Audience

This guide is intended for:

- Engineers and technical writers who author XML, DITA, or DocBook content and require change tracking, peer review, and reproducible publishing from a single source of truth.
- Contributors integrating documentation changes into repositories shared with software development teams.
- Reviewers and maintainers responsible for evaluating and merging documentation pull requests.

1.3 Prerequisites

Requirement	Specification
Oxygen XML Editor or Oxygen XML Author	Version 23.0 or later. Includes the embedded Git client; no external Git installation is required for the workflow described here.
Bitbucket repository	Bitbucket Cloud or a self-hosted Bitbucket Server/Data Center instance, with at least read access for cloning and write access for pushing branches and opening pull requests.
Git (optional)	A standalone Git installation is not required for the Oxygen-based workflow but is recommended if command-line operations are also performed.
Authentication credentials	An app password (Bitbucket Cloud) or SSH key pair, configured per Section 3.

Note: Git engine embedded with Oxygen XML Author is sufficient for all procedures in this guide. A standalone Git client is necessary only for command-line operations outside the editor.

2 Getting Started

This chapter walks through the minimum sequence of steps needed to go from a fresh installation of Oxygen XML Editor to your first committed change in Bitbucket. It is a condensed entry point into Sections 3 through 5; refer back to those sections for full detail on each step.

Before You Begin

Confirm the following are in place:

- Oxygen XML Editor or Oxygen XML Author, version 23.0 or later, is installed.
- You have at least read access to the target Bitbucket repository, and write access if you intend to push changes.
- You have generated a Bitbucket app password or SSH key (Section 3.1).

Step 1: Connect Oxygen to Bitbucket

Register your Bitbucket credentials once, under **Window > Preferences > Git > Credentials**.

This is a one-time setup per machine; Oxygen reuses the credentials for every repository on that Bitbucket host afterward. Full instructions are in Section 3.

Step 2: Clone the Repository

Open the **Git Staging** view (**Window > Show View > Git Staging**) and select **Clone repository**.

Provide the repository URL from Bitbucket and a local destination folder. Once cloning completes, open the project file (typically a DITA map or project.xpr) through **File > Open**. Full instructions are in [section 5](#).

Step 3: Make and Commit a Change

With the project open:

1. Pull the latest changes (**Pull** icon in the Git Staging view) to ensure you are starting from the current state of the repository.
2. Open a topic file and make an edit.
3. Stage the modified file, write a commit message describing the change, and select **Commit**.
4. Push the commit to Bitbucket (**Push** icon).

For more details, refer to [section 5](#).

3 Architecture: How the Three Components Interoperate

In this chapter:

[3.1 Rationale for a Git-Based Documentation Workflow](#)

[3.2 Rationale for Using Oxygen Native Git Integration](#)

Component	Function
Git	The version control system. Records every change as a commit, maintaining a complete, auditable history and enabling concurrent work via branches without overwriting conflicting changes.
Bitbucket	The remote repository host. Stores the canonical copy of the content, enforces access control and branch permissions, and provides the pull request and code review interface.
Oxygen XML Editor Git plugin	Editor Git plugin The client application. Provides clone, branch, stage, commit, push, and pull operations from within the authoring environment, in addition to XML-aware diff and three-way merge tooling.

The data flow is layered: Bitbucket is the system of record; Git defines the protocol and object model used to transfer and reconcile changes between Bitbucket and the local working copy; Oxygen is the client application through which the engineer interacts with both layers during authoring.

3.1 Rationale for a Git-Based Documentation Workflow

- Every change is attributable: each commit records the author, timestamp, and rationale.
- Concurrent work is supported through branch isolation, including parallel edits to the same topic.
- Changes are reviewable prior to integration, via pull requests, consistent with standard software review practices.
- Because DITA and XML source files are plain text, Git produces precise, line-level diffs rather than opaque binary comparisons.

3.2 Rationale for Using Oxygen Native Git Integration

- The diff and merge views are XML-structure-aware, comparing content at the element level rather than as unstructured text.
- Content can be validated prior to commit, reducing the likelihood of propagating malformed or invalid XML to the shared repository.

- Branch switching, staging, and committing are performed within the authoring environment, eliminating context-switching to a separate Git client.

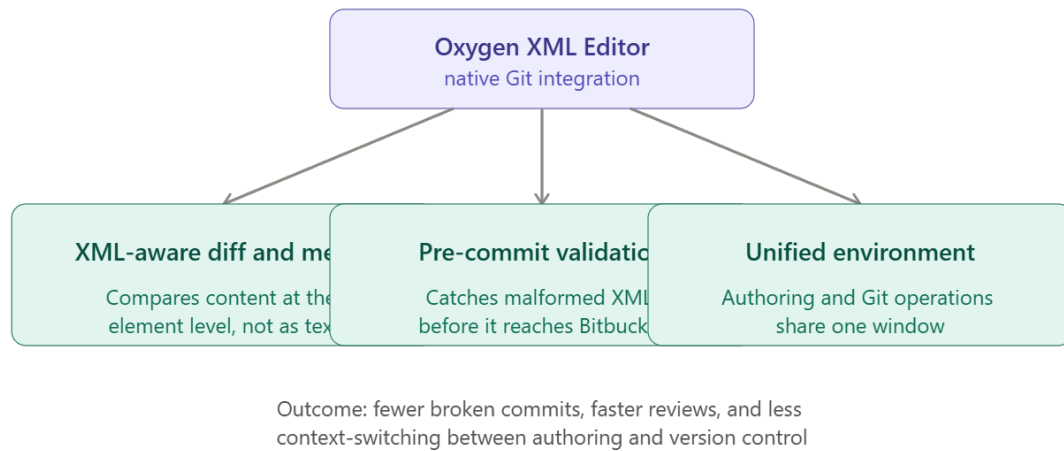


Figure 01 – Oxygen Git Integration benefits

4 Initial Configuration: Connecting Oxygen to Bitbucket

In this chapter:

[4.1 Generate Bitbucket Credentials](#)

[4.2 Register Credentials in Oxygen](#)

4.1 Generate Bitbucket Credentials

Bitbucket Cloud requires an app password or SSH key for Git operations; the account password is not accepted.

1. In Bitbucket, navigate to **Personal settings > App passwords**.
2. Select **Create app password** and grant **Repositories: Read** and **Write** permissions.
3. Copy the generated password immediately.
4. Bitbucket displays it once only.

Tip: If your team standardizes on SSH authentication, generate an SSH key pair and register the public key under **Personal settings > SSH keys** in Bitbucket. SSH authentication eliminates repeated credential prompts on push and pull operations.

4.2 Register Credentials in Oxygen

- Open **Window > Preferences > Git > Credentials**.
- Add an entry for the relevant Bitbucket host, specifying the account username and the app password generated in Section 4.1.
- Select **Apply**, then **OK** to persist the configuration.

Once registered, Oxygen authenticates automatically for all repositories hosted on that Bitbucket server; no further credential entry is required per repository.

5 Cloning a Documentation Repository

In this chapter:

[5.1 Clone via the Git Staging View](#)

[5.2 Open the Project](#)

5.1 Clone via the Git Staging View

1. Open the **Git Staging** view (**Window > Show View > Git Staging**).
2. Select the **Clone repository** icon, or **Git > Clone Repository** from the main menu.
3. Specify the repository URL, for example:

```
git clone https://github.com/your-username/hello-world.git
cd hello-world
```
4. Specify a local destination directory and select **Clone**.
5. Oxygen retrieves the complete repository, including all branches and commit history, and registers it in the Git Staging view.

Note: The repository URL is available on the Bitbucket repository overview page via the **Clone** control. Use the HTTPS URL for app-password authentication, or the SSH URL for key-based authentication.

5.2 Open the Project

After cloning, use **File > Open** to load the project entry point (a DITA map, project.xpr, or equivalent) from the cloned directory. The Project view and the Git Staging view then reference the same working copy, so file modifications made in the editor are immediately reflected in Git's change tracking.

6 Standard Editing Workflow

In this chapter:

[6.1 Pull Latest Changes](#)

[6.2 Edit Content](#)

[6.3 Review Modifications](#)

[6.4 Stage and Commit](#)

[6.5 Push to Bitbucket](#)

The recurring operational cycle in Oxygen mirrors the standard Git workflow: pull, edit, review, stage, commit, push.

6.1 Pull Latest Changes

Before beginning work, pull to synchronize the local branch with the remote. In the Git Staging view, select the **Pull** icon (down arrow). Oxygen fetches new commits from Bitbucket and merges them into the current local branch.

6.2 Edit Content

Open the target DITA topic, DocBook chapter, or other XML file in Author or Text mode and apply changes. On save, Oxygen detects the modification and lists the file under **Unstaged files** in the Git Staging view.

6.3 Review Modifications

Right-click a modified file in the Git Staging view and select **Show Diff** to open Oxygen's XML-aware comparison view, which renders additions, deletions, and structural changes at the element level.

6.4 Stage and Commit

1. In the Git Staging view, select the file(s) to include and select **Stage**, moving them to **Staged files**.
2. Enter a commit message that specifies the change and its rationale, for example:

Update installation topic with new prerequisites

3. Select **Commit**. The change is recorded in the local repository history.

Tip: Scope each commit to a single logical change (one topic, or one coherent set of related edits). This preserves a legible history and simplifies pull request review.

6.5 Push to Bitbucket

Select the **Push** icon (up arrow) in the Git Staging view to transmit committed changes to the corresponding branch in Bitbucket, making them visible to other contributors.

7 Branching Strategy for Documentation Changes

In this chapter:

[7.1 Create a Branch in Oxygen](#)

[7.2 Reference Branching Model](#)

Branches isolate in-progress content changes from the published or shared baseline, enabling parallel work without affecting other contributors or readers.

7.1 Create a Branch in Oxygen

- In the Git Staging view, select the current branch name to open the branch selector.
- Select **Create a new branch**.
- Specify a descriptive branch name (for example, update-install-guide) and confirm.
- Oxygen switches the working copy to the new branch; subsequent commits apply to it exclusively.

7.2 Reference Branching Model

Branch	Purpose
main (or master)	The published, stable documentation baseline.
develop	Optional integration branch where completed topic updates are aggregated prior to a release.
feature/<topic-name>	A short-lived branch scoped to a single topic update, new chapter, or correction, created from main or develop.

Note: Branch naming conventions are team-defined. The governing principle is that any substantial content change is isolated on its own branch, separate from the version currently visible to readers or other contributors.

8 Pull Request Review in Bitbucket

In this chapter:

[8.1 Open a Pull Request](#)

[8.2 Review Process](#)

[8.3 Merge](#)

A pull request is the mechanism by which a content change is proposed, reviewed, and approved prior to integration into the target branch.

8.1 Open a Pull Request

1. After pushing the feature branch, open the repository in Bitbucket.
2. Select **Create pull request**, specifying the feature branch as the source and main as the destination.
3. Provide a title and description summarizing the change, including a reference to any associated issue or ticket.
4. Assign one or more reviewers and select **Create**.

8.2 Review Process

- Reviewers examine a file-by-file diff in Bitbucket and may attach inline comments to specific lines.
- The author may respond to comments and push additional commits to the same branch; the pull request updates automatically to reflect new commits.
- A reviewer approves the pull request once outstanding comments are resolved.

8.3 Merge

On approval, select **Merge** in Bitbucket. Depending on repository configuration, this produces either a merge commit or a replay of the branch's commits onto main via squash or rebase. Following the merge, delete the feature branch and pull the updated main branch in Oxygen to synchronize the local working copy.

9 Conflict Resolution in XML Files

In this chapter:

[9.1 Identifying a Conflict](#)

[9.2 Resolution Procedure](#)

A conflict arises when two branches modify the same lines of a file in incompatible ways, and Git cannot reconcile the changes automatically. This condition is more probable in documentation repositories where multiple contributors edit a shared topic concurrently.

9.1 Identifying a Conflict

On pull or merge, Oxygen flags the affected file as **Conflicting** in the Git Staging view. The file itself contains standard Git conflict markers (<<<<<<, =====, >>>>>>) delimiting the competing changes.

9.2 Resolution Procedure

1. Double-click the conflicting file in the Git Staging view to open Oxygen's three-way XML merge editor.
2. Compare the **Your changes**, **Their changes**, and **Base** versions side by side.
3. Select the resolution for each conflicting element, or edit the merged result directly.
4. Validate the resulting XML to confirm well-formedness and, where applicable, schema conformance.
5. Stage and commit the resolved file to complete the merge.

Tip: Oxygen's merge view operates on XML structure rather than raw text, allowing it to align corresponding elements even when surrounding content has shifted. This produces more reliable resolutions than a plain-text merge tool.

10 Operational Best Practices

- Scope each commit to a single topic or a coherent set of related edits to preserve a legible, auditable history.
- Write commit messages that describe the substantive content change and its rationale (for example, `Clarify rollback steps in upgrade topic`), not generic descriptions such as `Edits`.
- Pull before beginning each editing session to minimize the likelihood of conflicts with concurrent work.
- Validate XML, and run any applicable DITA-OT or schema checks, prior to committing, to prevent malformed markup from entering the shared repository.
- Use short-lived feature branches for topic updates rather than committing directly to `main`.
- Route all but trivial changes through a pull request, including documentation-only repositories, to ensure independent review.
- Standardize branch naming and workflow conventions across the documentation team to maintain repository predictability.

11 Quick Reference

Task	Procedure
Clone repository	Git > Clone Repository in Oxygen; specify the Bitbucket URL
Pull latest changes	Select the Pull (down arrow) icon in the Git Staging view
View changes	Right-click a file in Git Staging; select Show Diff
Stage a file	Select the file in Git Staging; select Stage
Commit	Enter a commit message in Git Staging; select Commit
Push	Select the Push (up arrow) icon in the Git Staging view
Create a branch	Select the branch name; select Create a new branch
Open a pull request	In Bitbucket, select Create pull request on the repository page
Resolve a conflict	Double-click the conflicting file to open the XML merge editor

12 Glossary

Term	Definition
Repository	The complete set of documentation files and their full change history, hosted in Bitbucket.
Working copy	The local directory in which Oxygen maintains the files under edit.
Staging area	A Git holding area for changes selected for inclusion in the next commit.
Branch	An independent line of development used to isolate a set of content changes.
Pull request	A Bitbucket-mediated request to merge one branch into another, used to enforce review prior to integration.
Merge conflict	A condition in which Git cannot automatically reconcile changes from two branches affecting the same lines of a file.
DITA map	An XML file defining the structure and sequencing of topics within a DITA-based documentation set.
App password	A Bitbucket-issued credential used in place of the account password for Git operations.